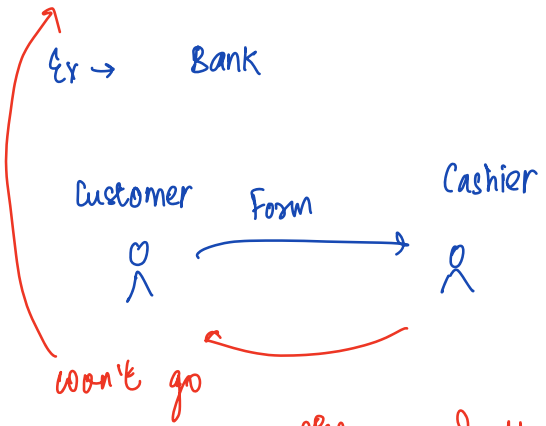
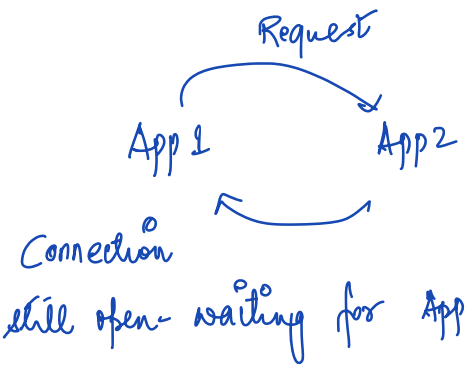
Synchronous Communication / Blocking Call.

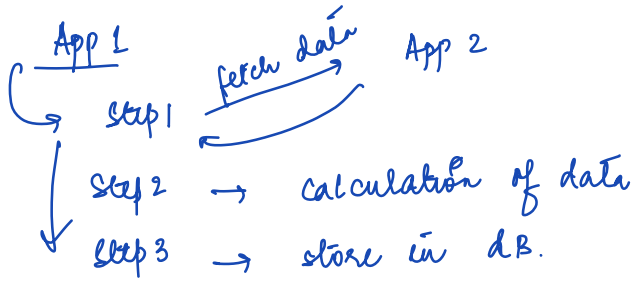
Ex → Bank





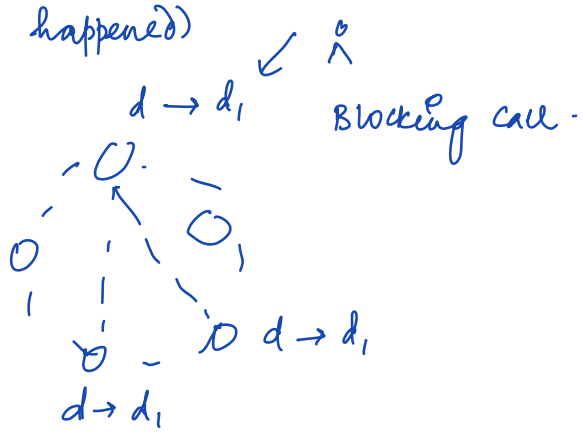
- Why we need it ?

→ Situation where we Immediatily need response.



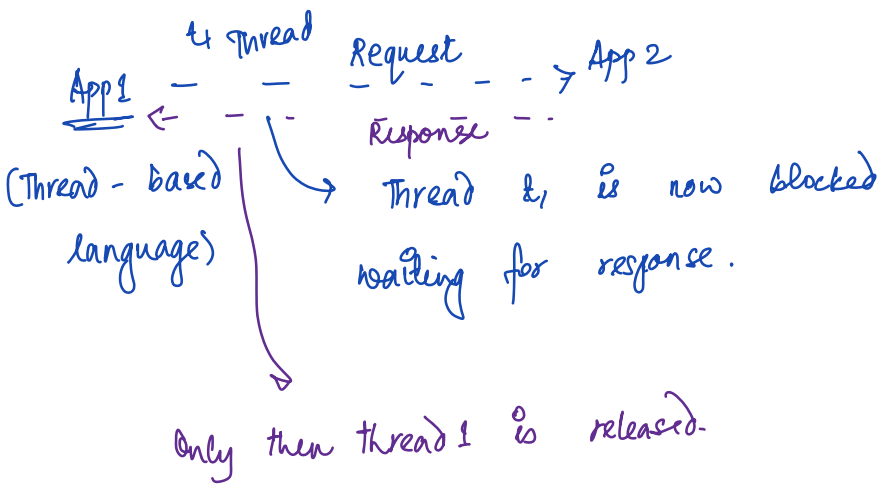
→ Transaction (Either everything has happened or nothing has happened)

→ Consistency



Blocking call.

How it is achieved?



→ Communication session is established by both sender & receiver.

Industrial Use Case

(a) Transaction → Banking → Phone Pe, Google Pay, UPI
→ Payments

(b) Reservation → IRCTC etc (Data has to be consistent)

(c) Sequential Apps (1) → (2) → (3)

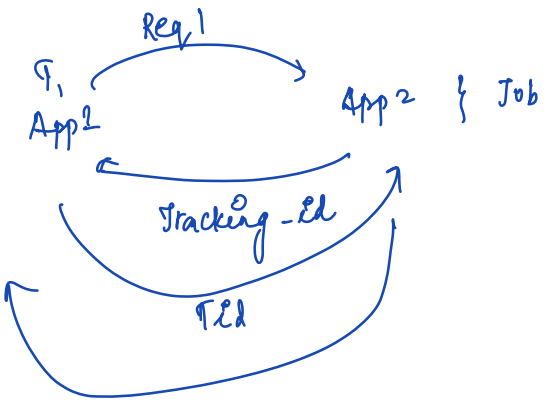
(d) Troubleshooting / Real Time decision making

(e) Stock Market

Asynchronous communication (Async call) / Non-Blocking / Start-Stop Communication

→ loosely coupled communication

eg Cake Shop – get cake after 3 hrs so customer comes after 3 hours (cake shop gives him tracking receipt) → customer gives that receipt & gets cake



Need → (a) When computation takes a lot of time

(b) scalability / Performance of the application.

Booking Application
- select Ticket
- Payment
- Booking Confirmation

} sync

- Email / SMS confirmation } async → this will have great impact on performance.

(c) App 1 ↓ Request 500 K/s

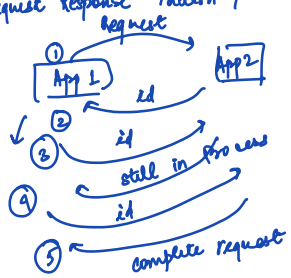
App 2 Receive 250 K/s } → cascading failure. (App will not be able to take more load) → as App 1 will put more & more load on App 2

ΔR = 250 K/s

(App 2 will crash) so here'll can have async communication

How it works?

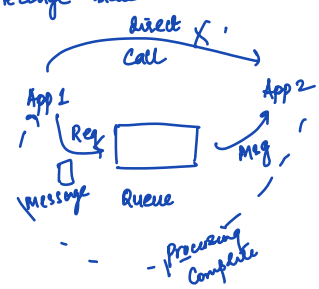
① Request Response Pattern / Tracking ID Based.



job → id

generally this pattern

② message queue Based Asyn communication

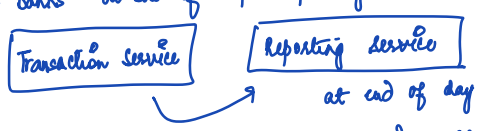


No direct connection b/w app1 & app2

Examples

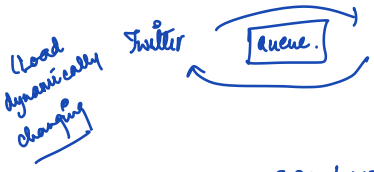
① E-commerce / Booking is involved – Confirmation Email / SMS / Notification

② Banks – at end of day – reporting



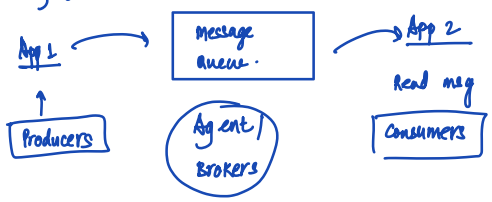
at end of day so can be done as async system

③ Applications who are reading tweets from Twitter & predicting sentiments



Message Queue → FIFO / LIFO

Why? → To allow communication b/w 2 apps.



eg Kafka – most highly used distributed messaging system

Rabbit MQ

Active MQ

Scenarios

① App

Throughput

500 GB/hr

Sudden Spike

eg Twitter – some celeb posts.

↳ 800 GB/hr

(app slows down → may even face cascading failure)

→ So how to design app?

↓

keep messaging queue before app.

↓

App can still take its own time can consume messages as per its throughput capacity & process.

② App 1 ← Updates

Message queue

App 2 consume

App 3

App 4

They should also get these updates

↓

If we can have some delays

advantage : App 5 also wants updates (Eventual Consistency)

↓

App 5 simply starts consuming.

(Diff Rate) (Diff Rate)

(3) App 1 communicate App 2 → further processing.

w/o knowing

(Processing) about each other (keeping them separate)

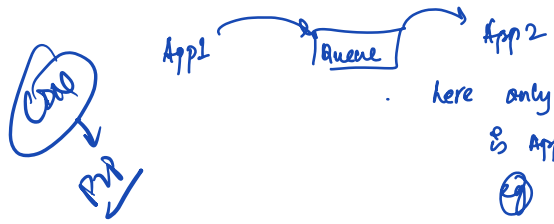
Pick the processed data

Can use message queue.

Failure of one will not impact 2.

Models of Message System

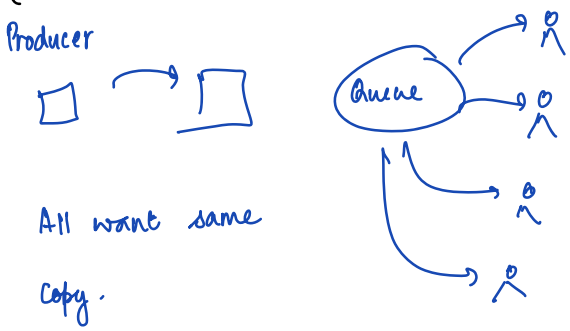
(a) P2P (Peer to Peer messaging)



. here only interested party is App2

eg we chat with somebody - we want our msg to be sent to that person.

(b) Publish Subscribe Model
(Pub-Sub Model)



All want same copy.

eg Newspaper

Industrial Examples

- Kafka → Messaging sol^n.
- Rabbit MQ
- Active MQ

Apache Kafka is a distributed event store and stream-processing platform. It is an open-source system developed by the Apache Software Foundation written in Java and Scala. Kafka is a tool which provides asynchronous communication to its users. Apache Kafka is used by thousands of companies for high-performance data pipelines, streaming analytics, data integration, and mission-critical applications.